MongoDB

Introduction to MongoDB

Definition: MongoDB is a cross-platform, open-source, **document-oriented NoSQL database**. It stores data as **BSON (Binary JSON)** documents within collections, supports dynamic schemas, and is built for horizontal scalability and high-throughput workloads.

MongoDB follows a strict hierarchy: **Database** → **Collection** → **Document** → **Field**.

MongoDB	RDBMS equivalent
Database	Database
Collection	Table
Document	Row
Field	Column

Unlike an RDBMS table, documents in the same collection *need not share the same fields*.

History and Evolution of MongoDB

- **2007:** 10gen founded by Dwight Merriman, Eliot Horowitz, and Kevin Ryan. The database component was designed to handle unstructured, web-scale data that relational systems could not accommodate cost-effectively.
- **2009:** MongoDB open-sourced. Introduced a document-oriented, JSON-like storage model as a direct departure from the relational paradigm.
- **2010–2012:** Rapid adoption driven by schema flexibility, horizontal scaling via sharding, and native support for hierarchical data. Early adopters included Foursquare and MTV Networks.
- **2013:** 10gen rebrands to **MongoDB, Inc.** Significant investment round accelerates enterprise growth.
- **2016:** MongoDB 3.4 adds read-only views and collation support. Enterprise features (auditing, LDAP auth) mature.
- **2017:** MongoDB, Inc. IPOs on Nasdaq (ticker: MDB).
- **2018:** MongoDB 4.0 introduces **multi-document ACID transactions** — the primary historical criticism addressed. Operations spanning multiple documents and collections can now be wrapped in all-or-nothing transactions.
- **2019–present:** MongoDB Atlas (fully managed cloud service) becomes the primary growth engine, supporting AWS, GCP, and Azure. Deployed at Adobe, eBay, Forbes, Toyota, Verizon.

Key Features of MongoDB

1. **Document-Based Storage:** Data stored as BSON documents containing nested objects and arrays. A product document can embed its reviews and specifications in one record, so a single `find()` retrieves everything without joins.
2. **Schema-Less Design:** Collections impose no fixed schema. One document can have a `phone` field; the next can omit it entirely. Example: an e-commerce app can store electronics with a `voltage` field and clothing with a `size` field in the same `products` collection.

3. **Horizontal Scalability via Sharding:** Data is split across shards using a *shard key*. A `mongos` router sits in front and directs each query to the correct shard. Example: an `orders` collection sharded on `customerId` distributes orders evenly across 10 servers — each server handles 1/10th the load.
4. **Replication (Replica Sets):** A replica set typically has one primary and two secondaries. If the primary crashes, the secondaries elect a new primary within seconds — the application reconnects automatically. Reads can be directed to secondaries to scale read throughput.
5. **Rich Query Language (MQL):** Supports filtering (`$eq`, `$gt`, `$in`), projection, sorting, pagination (`skip`/`limit`), and array operators. Example: `db.orders.find({ status: "shipped", total: { $gt: 500 } })` needs no SQL string construction.
6. **Indexing:** Without an index, MongoDB performs a *collection scan* — it reads every document. An index on `email` means a lookup by email checks the B-tree and jumps straight to the matching document. Supports compound, multi-key, text, TTL, and geospatial index types.
7. **Aggregation Framework:** Pipeline-based engine where documents flow through stages. Example: a sales report pipeline — `$match` filters to 2024 orders, `$group` sums revenue per product, `$sort` ranks by revenue. Each stage transforms the stream and passes it to the next.
8. **Multi-Document ACID Transactions:** Since v4.0, a bank transfer that debits one account and credits another can be wrapped in a transaction — both operations succeed or both are rolled back atomically.
9. **Document-Level Concurrency:** WiredTiger locks only the document being written, not the entire collection or database. Two writers updating different documents in the same collection proceed simultaneously with no blocking.
10. **Change Streams:** An application can call `db.orders.watch()` and receive a real-time event every time an order document changes — without polling. Built on the internal oplog.
11. **Cloud-Native (MongoDB Atlas):** Fully managed service on AWS, GCP, or Azure. Handles backups, upgrades, scaling, and monitoring. Example: Atlas auto-scales a cluster when CPU exceeds 75% without any manual intervention.
12. **Geospatial Support:** A `2dsphere` index on a `location` field enables queries like "find all restaurants within 2 km of this coordinate" using `$near` or `$geoWithin`.

Advantages of MongoDB

1. **Schema Flexibility:** No ALTER TABLE migrations when requirements change — critical in agile environments.
2. **Scalability:** Horizontal scaling via sharding handles traffic and data volumes beyond single-server limits without application changes.
3. **High Performance:** Embedded documents eliminate joins for common access patterns. WiredTiger document-level locking delivers fast concurrent reads and writes.
4. **Developer Productivity:** BSON maps directly to native objects (Python dicts, JavaScript objects). Less impedance mismatch than ORM-mediated relational access.
5. **Rich Query Capabilities:** MQL supports complex filtering, text search, geospatial queries, and aggregation without a separate analytics engine.
6. **ACID Transactions:** Multi-document transactions (v4.0+) provide integrity guarantees for financial and order-management use cases.
7. **Real-Time Analytics:** Change streams and the aggregation pipeline allow in-place analytics without ETL pipelines for many workloads.
8. **Cloud Integration:** Atlas simplifies deployment, eliminates DBA overhead, and enables

global, multi-region clusters.

9. **Security:** Built-in authentication (SCRAM, x.509), role-based access control, field-level encryption, TLS/SSL in transit.
10. **Big Data and IoT Support:** Schema flexibility and horizontal scaling accommodate heterogeneous, high-velocity data from IoT devices and analytics pipelines.

Key disadvantages: Denormalised model causes data duplication. Memory footprint is higher than many relational engines because field names are repeated in every document. Multi-document transactions add latency overhead compared to single-document writes.

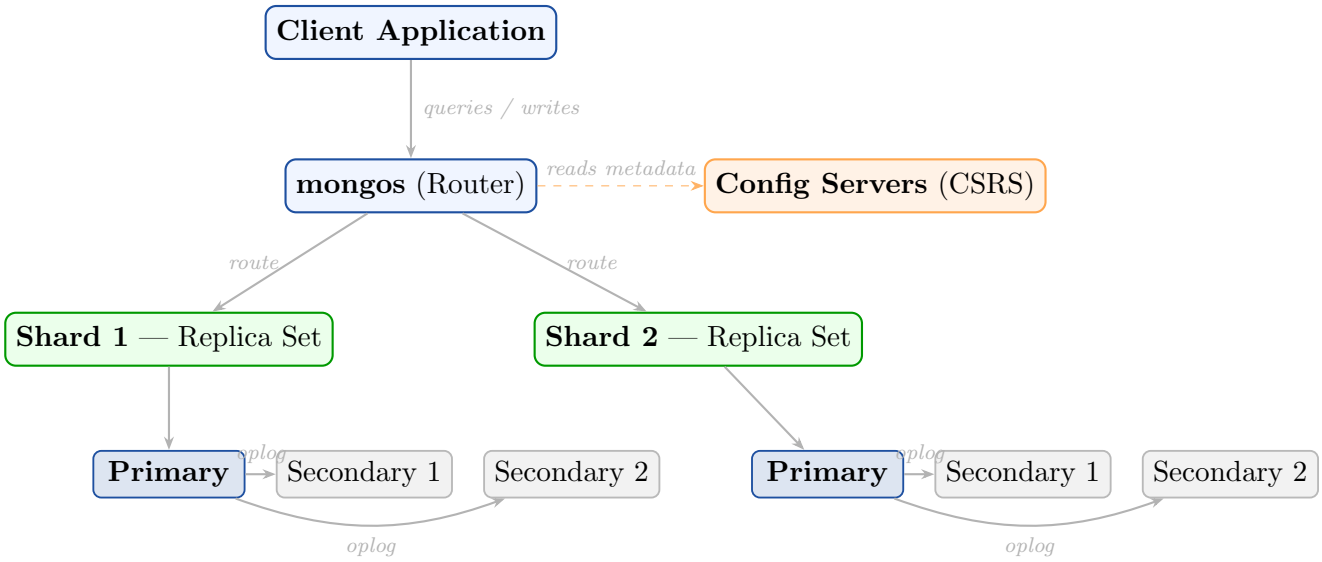
MongoDB Architecture

MongoDB's architecture is layered. At the top, client applications talk to a router. The router consults configuration metadata and forwards requests to the correct data servers. Each data server replicates its data to peers for redundancy.

Logical Hierarchy

Layer	Description
Client Application	Your app (web, mobile, backend service). Connects via a MongoDB driver.
mongos Router	Query router. Receives all client requests, routes to correct shard.
Config Servers	Replica set storing cluster metadata (chunk ranges, shard locations).
Shards	Data-bearing servers, each a replica set (primary + secondaries).

Architecture Diagram



Reading the diagram: The client never talks to a shard directly. All traffic goes through `mongos`. `mongos` consults config servers (dashed line) to find which shard owns the requested data chunk, then routes to that shard's primary. Within each shard, the primary replicates to secondaries via the oplog.

Component Responsibilities

Component	Responsibility
Client / Driver	Sends read/write requests. The driver handles connection pooling, retries, and failover transparently.
mongos	Stateless query router. Reads chunk metadata from config servers, merges results from multiple shards for scatter-gather queries. Multiple `mongos` instances can run in parallel for load balancing.
Config Servers	Store the cluster's routing table: which shard key ranges map to which shards. Always a replica set (3 nodes) for redundancy.
Shard (Primary)	Handles all write operations for its portion of data. Records every write to the oplog.
Shard (Secondary)	Tails the primary's oplog and replicates changes. Serves reads (if read preference allows). Participates in elections on primary failure.
MongoDB Engine	Per-shard process (`mongod`) that executes reads, writes, aggregations, and index operations using WiredTiger storage engine.

Document-Based Storage

Definition: Document-based storage is a database model where each record is a self-describing, semi-structured **document** (typically JSON or BSON). Documents can vary in structure within the same collection and can contain nested objects and arrays.

MongoDB stores documents in **BSON** format. BSON extends JSON with additional types (ObjectId, Date, Binary, Decimal128) and is *length-prefixed*, making it faster to scan and traverse than plain JSON text.

How Document Storage Differs from Row-Based Storage

In a relational database, data for one entity is often split across multiple tables joined by foreign keys. To get a product with its reviews and categories you run a multi-table JOIN — a costly operation at scale.

In MongoDB, the entire entity lives in one document. A single `find()` returns everything in one disk read. There is no JOIN cost.

RDBMS (row-based)	MongoDB (document-based)
Product in `products` table	Product + reviews + specs in one document
Reviews in separate `reviews` table	Embedded as an array inside the document
Retrieval requires JOIN across tables	Single `find()` — one I/O operation
Schema enforced by table definition	Each document is self-describing
Adding a new field = ALTER TABLE	Insert a document with the new field directly

Atomic unit: In MongoDB, the document is the atomic unit of storage and update. A write to a single document is always atomic — partial writes cannot be observed by other clients, even without a transaction.

Anatomy of a MongoDB Document

```
{
  "_id": ObjectId("507f191e810c19729de860ea"),
  "name": "Wireless Headphones",
  "brand": "SoundCo",
  "price": 89.99,
  "inStock": true,
  "features": {
    "batteryLife": "20 hours",
    "connectivity": "Bluetooth 5.0"
  },
  "ratings": [
    { "userId": "u123", "score": 5, "comment": "Excellent" },
    { "userId": "u456", "score": 4, "comment": "Good value" }
  ],
  "created_at": ISODate("2024-01-15T10:30:00Z")
}
```

- `_id` — Primary key, auto-generated as a 12-byte ObjectId. Unique per collection.
- `features` — Embedded (nested) document. Fetched in the same read as the parent; no join required.
- `ratings` — Array of embedded documents. Supports `$elemMatch`, `$push`, `$pull` operators.

Characteristics of Document-Based Storage

- **Schema flexibility:** Documents can have different fields, accommodating evolving requirements without migrations.
- **Nested data:** Arrays and embedded documents represent complex relationships natively.
- **Powerful querying:** Unlike key-value stores, the DB understands document structure and supports querying on any field, including nested fields.

Use Cases

Domain	Why document storage fits
E-commerce	Product catalogues with varying attributes per category; order + items in one document
CMS	Articles, blog posts, media with differing metadata schemas
IoT	Heterogeneous sensor payloads that evolve across device generations
Social networks	User profiles, posts, comments — naturally hierarchical
Gaming	Player state, inventory, leaderboards — high write velocity
Financial services	Transaction records with variable compliance fields
Real-time analytics	High-throughput ingestion with inline aggregation

MongoDB Data Types

MongoDB stores all data as **BSON**. The following are the primary BSON data types:

Type	BSON code	Description / Example
String	2	UTF-8 text. `"name": "Alice"`
32-bit Integer	16	Whole number, 4 bytes. `"age": 25`
64-bit Integer	18	Larger whole number, 8 bytes.
Double	1	64-bit IEEE 754 float. `"price": 99.99`
Decimal128	19	High-precision decimal for financial data.
Boolean	8	`true` or `false`. `"isActive": true`
Array	4	Ordered list of values. `"tags": ["db", "nosql"]`
Object (Embedded doc)	3	Nested BSON document. `"address": {...}`
ObjectId	7	12-byte unique identifier, auto-generated `_id`.
Date	9	UTC ms since Unix epoch. `ISODate("2024-01-01")`
Null	10	Explicit absence of value. `"middleName": null`
Binary	5	Raw binary data (images, files).
Regular Expression	11	Stored regex pattern for query matching.
Timestamp	17	Internal replication timestamp. 64-bit value.

ObjectId structure: The 12 bytes encode (4 bytes) Unix timestamp + (5 bytes) random value + (3 bytes) incrementing counter. ObjectIds are roughly time-ordered and can be used to infer document insertion order.

Indexing in MongoDB

Definition: An index is a separate data structure (B-tree) that holds a sorted copy of one or more fields from a collection, along with pointers to the corresponding documents. Indexes allow MongoDB to locate matching documents without scanning every document in a collection.

Without an Index vs With an Index

Without an index (collection scan): MongoDB reads every document in the collection from disk, checks whether it matches the query filter, and returns matches. For a collection with 10 million documents, that is 10 million reads regardless of how many documents match.

With an index: MongoDB traverses the B-tree (O(log n) operations) to find the matching keys, then follows the pointers directly to the target documents. Only the matching documents are read.

You can see which path MongoDB takes using `.explain("executionStats")`. Look for `COLLSCAN` (bad — collection scan) vs `IXSCAN` (good — index scan) in the output.

Types of Indexes

Index Type	Description and Example
Single-field	Index on one field. `db.users.createIndex({ email: 1 })` — speeds up lookups by email. `1` = ascending, `-1` = descending.
Compound	Index on multiple fields. `db.orders.createIndex({ customerId: 1, date: -1 })` — speeds up queries filtering by customer and sorting by date together.
Multi-key	Automatically created when indexing an array field. Each array element gets its own index entry. `db.products.createIndex({ tags: 1 })` indexes every tag value.
Text	Enables full-text search on string fields. `db.articles.createIndex({ body: "text" })` allows `$text: { $search: "mongodb tutorial" }` queries.
TTL (Time-To-Live)	Auto-deletes documents after a set time. `db.sessions.createIndex({ createdAt: 1 }, { expireAfterSeconds: 3600 })` deletes session docs 1 hour after creation.
Geospatial (2dsphere)	Enables location-based queries on GeoJSON data. `db.places.createIndex({ location: "2dsphere" })` allows `$near` and `$geoWithin` queries.
Hashed	Hashes the field value for even distribution in sharded clusters. `db.users.createIndex({ userId: "hashed" })`.

Creating and Managing Indexes

```
// Create a single-field index on 'email' (ascending)
db.users.createIndex({ email: 1 })

// Create a compound index
db.orders.createIndex({ customerId: 1, orderDate: -1 })

// Create a TTL index (auto-expire after 1 hour)
db.sessions.createIndex({ createdAt: 1 }, { expireAfterSeconds: 3600
    })

// List all indexes on a collection
db.users.getIndexes()

// Analyse a query - look for IXSCAN vs COLLSCAN
db.users.find({ email: "a@b.com" }).explain("executionStats")
```

```
// Drop a specific index
db.users.dropIndex({ email: 1 })
```

Advantages and Disadvantages of Indexing

Advantages	Disadvantages
Dramatically faster reads — O(log n) instead of O(n) collection scan	Each index consumes additional disk space
Supports efficient sorting without in-memory sort	Every write (insert, update, delete) must also update all relevant indexes — slower writes
Enables covered queries (result served from index alone, no document fetch)	Over-indexing wastes memory; indexes are loaded into RAM (WiredTiger cache)
TTL indexes automate document expiry with no application code	Compound index field order matters; wrong order means the index is not used
Geospatial indexes enable location queries not possible otherwise	Multi-key indexes on large arrays can become very large and slow to update

Rule of thumb: Index fields that appear in `find()` filters, sort operations, and join conditions (`$lookup` foreign key). Do not index fields that are rarely queried or that have very low cardinality (e.g., a boolean `isActive` on a collection where 99% of documents are `true`).

Data modeling in MongoDB is driven by **access patterns**, not normalisation rules. The goal is to structure documents so that the most frequent queries are answered in a single round-trip without joins.

Embedding vs. Referencing

Aspect	Embedding	Referencing
Definition	Related data inside the same document	Related data in a separate document; linked by an ID
Read performance	One query fetches all data (no join)	Requires second query or `$lookup`
Write performance	Entire document rewritten on update	Only the referenced document updated
Document size	Can grow large; 16 MB BSON limit applies	Keeps documents lean

Aspect	Embedding	Referencing
Use when	Data always accessed together; bounded 1:N	Data is large, updated independently, or shared across many parents

Relationship Patterns

- **One-to-One:** Embed as a nested field if accessed together; reference if accessed independently.
- **One-to-Many (bounded):** Embed the array of children inside the parent (e.g., order items inside an order). Most common MongoDB pattern.
- **One-to-Many (unbounded):** Reference from child to parent (e.g., comments store a `postId`) to avoid unbounded document growth.
- **Many-to-Many:** Arrays of references on both sides, or an intermediate collection, depending on query pattern.

Schema Design Best Practices

1. **Design for queries, not entities:** Ask "what will the application query?" before deciding on document structure.
2. **Favour embedding for data read together:** Embedded subdocuments are fetched atomically in one I/O operation.
3. **Avoid unbounded arrays:** Arrays that grow indefinitely approach the 16 MB document limit and degrade index performance.
4. **Denormalise strategically:** Duplicate fields that are read frequently but updated rarely to avoid joins.
5. **Bucket Pattern for time-series:** Group measurements into documents (e.g., one per hour per sensor) to reduce document count and improve range query performance.

Example: E-Commerce Order Model

```
{
  "_id": ObjectId("..."),
  "orderId": "ORD-12345",
  "customerId": ObjectId("c123"),    // reference to Customer
     collection
  "orderDate": ISODate("2024-03-01"),
  "status": "shipped",
  "items": [                          // embedded (bounded 1:N)
    { "productId": "p101", "name": "Laptop", "qty": 1, "price": 1000
        },
    { "productId": "p102", "name": "Mouse",  "qty": 2, "price": 20
         }
  ],
  "totalAmount": 1040
}
```

Order items are embedded because they are always accessed with the order and never exceed a bounded count. The customer is referenced because the profile is updated independently and referenced from many orders.

Aggregation Framework

Definition: The aggregation framework is MongoDB's built-in data processing engine. It processes documents through a **pipeline** — a sequence of stages where each stage transforms the stream of documents and passes the result to the next stage. Used for grouping, filtering, computing totals, and reshaping documents.

Mental model: Like a factory assembly line — raw documents enter at one end, each stage does one job (filter, group, sort, reshape), and the finished result exits at the other end.

Key Pipeline Stages

Stage	What it does
`$match`	Filters documents — only matching ones pass to the next stage. Equivalent to `find()`. Place first to reduce the working set and leverage indexes.
`$group`	Groups documents by a key and computes per-group aggregations: `$sum`, `$avg`, `$count`, `$min`, `$max`.
`$sort`	Sorts the document stream. `1` = ascending, `-1` = descending.
`$project`	Reshapes documents — include, exclude, or compute new fields.
`$limit` / `$skip`	Keep first N / skip first N documents. Used together for pagination and top-N queries.
`$lookup`	Left outer join with another collection. Adds matching documents as an array field.
`$unwind`	Deconstructs an array field into one document per element. Typically used before `$group` to aggregate on array values.

Example

Count completed orders grouped by status and sort by count:

```
db.orders.aggregate([
  { $match:  { status: "completed" } },
  { $group:  { _id: "$category", total: { $sum: 1 } } },
  { $sort:   { total: -1 } }
])
```

How Aggregation Enhances Performance

- **$match uses indexes:** When `$match` is the first stage and the filter field is indexed, only matching documents enter the pipeline — the rest are never read.
- **Server-side computation:** Grouping and arithmetic run inside the database process, not in application code, eliminating the need to transfer large datasets to the client.
- **No separate analytics tool needed:** Reports that would otherwise require an ETL pipeline to a data warehouse can be computed directly on the live collection.

Installation Options

- **Community Edition:** Free, open-source. Installed via `apt`, `yum`, Homebrew, or binary download.
- **Enterprise Edition:** Adds auditing, encrypted storage engine, LDAP authentication.
- **MongoDB Atlas:** Fully managed cloud deployment. No local installation required. Recommended for cloud-native applications.

Core Processes

- `mongod` — Primary daemon. Handles storage, queries, replication, and sharding. Default port: **27017**.
- `mongos` — Query router in a sharded cluster. Directs client requests to the correct shard.
- `mongosh` — Interactive shell client (installed separately from the server).

Scalability and Performance in MongoDB

MongoDB is designed so that as your data or traffic grows, you scale *out* (add more machines) rather than *up* (buy a bigger machine). Three mechanisms work together: sharding, replication, and indexing.

Vertical vs Horizontal Scaling

Aspect	Vertical Scaling (Scale-Up)	Horizontal Scaling (Scale-Out)
Definition	Increasing the capacity of a single server — more CPU, RAM, or faster storage.	Adding more servers to the system and distributing the load across them.
How	Upgrade hardware on the existing machine.	Add new commodity machines to the cluster.
Cost	Expensive; high-end hardware costs grow non-linearly.	Cost-effective; uses many cheap, standard servers.
Ceiling	Hard limit — there is a maximum to how powerful one machine can be.	Virtually unlimited; keep adding nodes as needed.
Downtime	Usually requires downtime to upgrade hardware.	New nodes can be added with zero downtime.
Failure risk	Single point of failure — if the one server goes down, everything goes down.	Fault-tolerant — other nodes continue serving traffic.
MongoDB fit	Not MongoDB's primary model; possible but limited.	MongoDB's native model via sharding and replica sets.
Typical use	Traditional RDBMS (Oracle, MySQL).	Distributed NoSQL systems (MongoDB, Cassandra).

Why this matters for MongoDB: MongoDB's entire architecture — shard keys, mongos routers, config servers, replica sets — is designed around horizontal scaling. This is the core reason it was built: to handle workloads that no single machine, no matter how powerful, could serve alone.

Sharding (Horizontal Scaling)

Sharding is the process of distributing data across multiple servers (shards). Each shard holds a mutually exclusive subset of the collection's data. Together, all shards hold the complete dataset. Sharding is MongoDB's solution for collections that grow beyond what a single server can store or serve efficiently.

Why sharding is needed:

- A single server has finite RAM, CPU, and disk. At scale (billions of documents), a single node becomes a bottleneck.
- Vertical scaling (bigger hardware) is expensive and has a hard ceiling.
- Sharding spreads both storage and query load across many commodity machines — costs scale linearly, not exponentially.

Components of a sharded cluster:

Component	Role
Shard	A single MongoDB instance (or replica set) that stores a portion of the sharded collection's data.
Shard Key	A field (or compound fields) chosen by the developer. MongoDB uses its values to decide which shard each document belongs to. Choosing a good shard key is critical — a bad one causes uneven distribution (hotspots).
Chunks	MongoDB divides the shard key value range into contiguous **chunks** and assigns each chunk to a shard. Default chunk size is 128 MB. When a chunk grows too large, MongoDB splits it automatically.
mongos Router	A lightweight process that acts as the query router. The application connects to `mongos`, never directly to shards. `mongos` inspects each query, consults config servers, and forwards to the correct shard(s).
Config Servers	Store the cluster's metadata: which chunks live on which shards, shard key ranges, and cluster topology. Always deployed as a replica set (CSRS).

How a sharded query works:

1. Application sends a query to `mongos`.
2. `mongos` checks config servers to find which shard(s) hold the relevant chunk(s).
3. `mongos` forwards the query to the target shard(s).
4. Results are merged by `mongos` and returned to the application.
5. If the query includes the shard key, only one shard is queried (**targeted query**). If not, all shards are queried (**scatter-gather query**) — less efficient.

Shard key strategies:

Strategy	How it works	Trade-off
Range-based	Documents with adjacent shard key values go to the same shard.	Efficient range queries; risk of hotspots if inserts are sequential (e.g., timestamps).
Hashed	The shard key value is hashed before assignment, distributing documents evenly.	Even distribution; range queries require scatter-gather.
Zone-based	Documents are assigned to specific shards based on shard key ranges mapped to zones.	Useful for data locality (e.g., EU user data on EU servers).

```
// Step 1: Enable sharding on the database
sh.enableSharding("ecommerce")

// Step 2: Shard a collection using a hashed shard key
sh.shardCollection("ecommerce.orders", { customerId: "hashed" })

// Step 3: Check sharding status
sh.status()
```

Effect: 100 million orders sharded across 10 servers — each server handles 10 million. Adding an 11th server triggers automatic chunk migration (balancing) with no downtime and no application changes.

Shard key is permanent. Once a collection is sharded, the shard key cannot be changed without resharding the entire collection. Choosing it carefully upfront — based on cardinality, write distribution, and query patterns — is one of the most important MongoDB design decisions.

Replication (High Availability + Read Scaling)

Replication is the process of synchronising data across multiple MongoDB servers to ensure the same dataset exists in more than one place. The unit of replication in MongoDB is a **replica set** — a group of `mongod` instances that all maintain the same data.

Why replication matters:

- **High availability:** If the primary server crashes, the system automatically recovers without manual intervention.
- **Data redundancy:** Multiple copies of data on different machines mean no single hardware failure can cause data loss.
- **Read scaling:** Read traffic can be distributed across secondaries, reducing load on the primary.

Replica set members:

Member	Role
Primary	The only node that accepts write operations. All inserts, updates, and deletes go here first. There is exactly one primary at any time.
Secondary	Replicates data from the primary by replaying entries from the primary's **oplog** (operations log). Can serve read requests if configured. A replica set typically has two or more secondaries.
Arbiter	A lightweight member that holds no data. Participates only in elections to break ties. Used when an even number of data-bearing nodes would make a majority impossible.

How replication works — the oplog:

Every write operation on the primary is recorded in a capped collection called the **oplog** (`local.oplog.rs`). Secondaries continuously tail the primary's oplog and replay each operation on their own copy of the data. This is asynchronous — secondaries may lag slightly behind the primary.

```
// Initiate a 3-member replica set (run on each node's mongosh)
rs.initiate({
  _id: "myReplicaSet",
  members: [
    { _id: 0, host: "mongo1:27017" },   // primary (initially)
    { _id: 1, host: "mongo2:27017" },   // secondary
    { _id: 2, host: "mongo3:27017" }    // secondary
  ]
})

// Check replica set status
rs.status()
```

Automatic failover:

If the primary becomes unreachable, the secondaries detect the failure via heartbeat timeouts (default: 10 seconds). The remaining members hold an **election** — each candidate requests votes from peers, and the node with the most up-to-date oplog and majority of votes wins. The new primary is elected typically within 10–15 seconds. The application driver detects the new primary automatically and resumes writes with no code changes.

Majority rule: A write is acknowledged as committed only once it has been replicated to a majority of replica set members (write concern `majority`). This prevents data loss if the primary fails immediately after a write. A 3-member set requires 2 acknowledgements; a 5-member set requires 3.

Read preferences:

By default, all reads go to the primary. This can be changed per-operation:

Read Preference	Behaviour
`primary` **(default)**	Always reads from primary. Guarantees the most recent data.
`primaryPreferred`	Reads from primary if available; falls back to secondary.
`secondary`	Always reads from a secondary. Data may be slightly stale.
`secondaryPreferred`	Reads from secondary if available; falls back to primary. Used to offload read traffic.
`nearest`	Reads from the member with the lowest network latency, regardless of role.

Replication vs Sharding — key distinction:

Replication	Sharding
Same data on multiple nodes	Different data on different nodes
Solves availability and redundancy	Solves data volume and write throughput
All nodes hold the full dataset	Each shard holds a subset
Protects against node failure	Allows horizontal scale-out

In production, MongoDB uses **both** together: each shard in a sharded cluster is itself a replica set. So data is both distributed (sharding) and redundant (replication) simultaneously.

WiredTiger Storage Engine Performance

WiredTiger (MongoDB's default storage engine since v3.2) contributes to performance through:

- **Document-level locking:** Writers lock only the document they are updating. Different documents in the same collection can be written simultaneously — no table-level lock contention.
- **Compression:** WiredTiger compresses data on disk (Snappy by default, zstd available). A collection that takes 100 GB uncompressed might use 30–50 GB on disk.
- **In-memory cache:** WiredTiger maintains a configurable in-memory cache (default: 50% of RAM). Frequently accessed documents are served from RAM, not disk.

Summary: Scalability Mechanisms

Mechanism	Solves	Example
Sharding	Data volume + write throughput	1 billion user records split across 20 shards
Replica sets	Read throughput + fault tolerance	3 replicas serve reads while primary handles writes
Indexing	Query latency	Find a user by email in O(log n) instead of O(n)
WiredTiger cache	Repeated access latency	Hot product pages served from RAM
Embedded documents	Join elimination	Fetch order + items in one query

MongoDB Shell (mongosh)

Definition: `mongosh` is the official interactive JavaScript/REPL interface for MongoDB. It connects to a running `mongod` instance and allows issuing database commands, running queries, managing collections, and executing JavaScript scripts.

Internally, `mongosh` communicates with `mongod` over the **MongoDB Wire Protocol** — a TCP-based binary protocol. Commands are serialised to BSON, sent over the wire, and responses are deserialised and pretty-printed.

Key Capabilities

- **Full JavaScript environment:** Variables, loops, functions, and async/await are valid. Enables scripting and automation directly from the shell.
- **Database and collection management:** `use <db>`, `show dbs`, `show collections`, `db.createCollection()`, `db.collection.drop()`.
- **CRUD operations:** `insertOne()`, `insertMany()`, `find()`, `updateOne()`, `deleteMany()`.
- **Query analysis:** `.explain("executionStats")` shows the query plan and index usage — essential for performance tuning.
- **Aggregation pipeline:** Full pipeline execution via `db.collection.aggregate([...])`.
- **Index management:** `createIndex()`, `getIndexes()`, `dropIndex()`.

MongoDB Shell vs MongoDB Compass

Aspect	mongosh (Shell)	MongoDB Compass
Interface	CLI / REPL	GUI
Query input	JavaScript / MQL	Visual query builder + MQL
Automation	Scriptable (JavaScript files)	Not scriptable
Index management	Full control via commands	Visual index creation
Schema exploration	Manual queries required	Visual schema analysis tool

Aspect	mongosh (Shell)	MongoDB Compass
Aggregation pipeline	Text pipeline syntax	Interactive pipeline builder
Learning curve	Higher (requires MQL knowledge)	Lower (point-and-click)
Best for	Developers, automation, production	Exploratory analysis, learning

Database: Create and Drop

MongoDB uses a **lazy creation** model: a database does not physically exist until the first document is inserted into a collection within it.

Creating a Database

```
use myDatabase                                  // switch context (DB
   not created yet)

db.myCollection.insertOne({ name: "Alice", age: 30 })
// database is now physically created and will appear in show dbs

show dbs                                        // verify myDatabase
   appears
```

Dropping a Database

```
use myDatabase
db.dropDatabase()       // permanently deletes all collections and
   data

show dbs                // myDatabase no longer appears
```

Dropping a database is irreversible. Always confirm the current database with `db` (prints current DB name) before executing `dropDatabase()`.

Useful Database Commands

```
db              // print current database name
show dbs        // list all databases with storage size
db.stats()      // storage statistics for current database
```

Collection: Create and Drop

Definition: A collection is a grouping of MongoDB documents, analogous to a table in RDBMS. Collections do not enforce a schema — documents within the same collection may differ in structure. Each collection belongs to exactly one database.

Creating a Collection

Implicit Creation (most common)

MongoDB creates a collection automatically when the first document is inserted:

```
db.students.insertOne({ name: "Ravi", grade: "A" })
// 'students' collection created automatically
```

Explicit Creation

Use `createCollection()` when you need to set options before inserting data:

```
db.createCollection("students")

// Capped collection (fixed size, FIFO eviction)
db.createCollection("logs", {
  capped: true,
  size: 5242880,     // 5 MB max
  max: 5000          // max 5000 documents
})

// With schema validation
db.createCollection("users", {
  validator: {
    $jsonSchema: {
      required: ["name", "email"],
      properties: {
        email: { bsonType: "string" }
      }
    }
  }
})
```

Dropping a Collection

```
db.students.drop()         // returns true if dropped, false if not
   found
show collections           // verify removal
```

Listing Collections

```
show collections
db.getCollectionNames()    // returns array of collection name
   strings
```

Dropping a collection removes all documents, all indexes, and all metadata permanently. It is faster than `deleteMany({})` because it removes the underlying data files directly rather than iterating through documents.

Collection vs Table

Aspect	MongoDB Collection	RDBMS Table
Schema enforcement	None by default (optional via validator)	Strict, predefined
Row structure	Documents may differ in fields	All rows share the same columns
Creation	Implicit on first insert	Must be explicitly defined (DDL)
Deletion	`.drop()`	`DROP TABLE`
Join support	Via `$lookup` in aggregation	Native SQL JOIN

Impact of Schema-Less Design on Data Management

What Schema-Less Means in Practice

In a relational database, the schema is defined before data is inserted. Every row must conform to the column definitions. Adding a new field requires an ALTER TABLE statement, which can lock the table and take hours on large datasets.

In MongoDB, there is no such contract. You insert a document with any fields you want. Two documents in the same collection can look completely different.

Positive Impacts

1. **Faster iteration:** A new feature requiring a new field is deployed by just writing documents with that field. No migration script, no downtime.
2. **Handles heterogeneous data:** A `products` collection can store laptops (with `RAM`, `processor`) and shirts (with `size`, `colour`) without a separate table for each type.
3. **Evolutionary design:** Applications can add, rename, or restructure fields over time without a schema version management system.
4. **Reduced development friction:** Developers work with the same JSON-like structure in code and in the database — no ORM mapping layer needed.

Negative Impacts and Risks

1. **Type inconsistency:** Without enforcement, one document might store `age` as a string (`"25"`) and another as an integer (`25`). Queries and aggregations that assume a type will break or return wrong results.
2. **Missing required fields:** Nothing stops an insert of a user document without an `email` field. Application logic must check for this.
3. **Data quality degrades over time:** Different developers writing different code paths may create documents with inconsistent field names (`phoneNumber` vs `phone_number` vs `phone`).
4. **Application-level validation burden:** Validation that a relational database enforces at the engine level (NOT NULL, CHECK constraints) must be written and maintained in application code.
5. **Querying incomplete data:** A field that is missing from some documents returns `null` or is omitted from results — the application must handle this case explicitly.

Mitigation: Schema Validation

MongoDB supports optional **JSON Schema validation** on collections to enforce rules while keeping flexibility:

```
db.createCollection("users", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "email", "age"],
      properties: {
        name:  { bsonType: "string" },
        email: { bsonType: "string", pattern: "^.+@.+$" },
        age:   { bsonType: "int", minimum: 0 }
      }
    }
  },
  validationAction: "error"   // reject invalid inserts
})
```

This gives the benefits of schema-less flexibility (optional fields are still allowed) while catching the most dangerous inconsistencies.

Exam angle: Schema-less is not "schema-free" in well-run applications. Mature MongoDB projects define an application-level schema (often using an ODM like Mongoose in Node.js), use MongoDB's validator for critical fields, and version their data models explicitly.

Real-World Applications: MongoDB vs RDBMS

The choice of MongoDB over RDBMS is justified when the data has properties that relational databases handle poorly: variable structure, hierarchical nesting, high volume, or rapidly evolving schemas.

Domain	Why MongoDB fits	Why RDBMS struggles
E-commerce product catalogue	Products vary in attributes (electronics have voltage, clothing has size). One document per product embeds all variants.	Every new product type requires ALTER TABLE or nullable columns for every other product type.
Content management (CMS)	Articles, videos, podcasts, and galleries each have different metadata. Schema-less documents handle all types in one collection.	Separate tables per content type, or a complex EAV (Entity-Attribute-Value) pattern that is slow and hard to query.
Social media / user profiles	User profiles grow over time (add bio, add location, add preferences). Embedding posts and activity is natural.	Adding profile fields requires schema migrations. Storing activity requires JOIN-heavy queries.

Domain	Why MongoDB fits	Why RDBMS struggles
IoT sensor data	Each device model emits different fields. Data volumes are massive and write rates are high.	Fixed schema breaks when a new sensor type is deployed. Single-node RDBMS cannot sustain the write rate.
Real-time analytics	Aggregation pipeline runs analytics in-database. Change streams feed dashboards in real time.	Real-time analytics typically requires a separate OLAP database or data warehouse.
Gaming (player state)	Player inventory, achievements, settings — all naturally nested in one document. High-velocity reads and writes at scale.	Normalising inventory into tables requires many JOINs per game tick. Vertical scaling hits limits quickly.
Mobile backends	Flexible document model matches the JSON payloads mobile apps send and receive. Atlas App Services and Atlas Device Sync enable offline-first mobile patterns.	Schema rigidity creates friction with evolving mobile app requirements.

When RDBMS is the better choice: Financial transaction ledgers requiring strict referential integrity, multi-entity transactional workflows (e.g., double-entry bookkeeping), reporting systems that depend on complex multi-table JOINs, and any domain where data relationships are more important than data volume.

Question Coverage Checklist — Module 02

Check off each question once you can answer it from memory.

Q No.	Question	Marks
Q1	What is MongoDB?	2
Q2	Define NoSQL database.	2
Q3	List any two key features of MongoDB.	2
Q4	What is document-based storage in MongoDB?	2
Q5	How can you create a database in MongoDB?	2
Q6	Mention any two MongoDB data types.	2
Q7	What is the MongoDB shell?	2
Q8	Define a collection in MongoDB.	2
Q9	How do you drop a collection in MongoDB?	2
Q10	Name two advantages of MongoDB over traditional RDBMS.	2
Q11	Explain the history and evolution of MongoDB.	4
Q12	Differentiate between NoSQL and SQL databases.	4
Q13	Illustrate the key features of MongoDB with examples.	4
Q14	How does document-based storage work in MongoDB?	4
Q15	Explain the process of creating and dropping a database in MongoDB.	4
Q16	Discuss the various MongoDB data types with examples.	4
Q17	Write the MongoDB commands to create and drop a collection.	4
Q18	Compare MongoDB shell and MongoDB Compass.	4
Q19	What are the advantages of using MongoDB in modern applications?	4
Q20	Explain the role of data modeling in MongoDB.	4
Q21	Discuss the importance of NoSQL databases in handling big data and real-time applications.	6
Q22	Evaluate the advantages and disadvantages of MongoDB compared to traditional relational databases.	6
Q23	Design a simple database structure for an e-commerce application using MongoDB and explain its components.	6
Q24	How does MongoDB handle scalability and performance? Explain with suitable examples.	6
Q25	Create a data model for a student management system in MongoDB and justify your choices.	6
Q26	Analyze the impact of MongoDB's schema-less nature on data management.	6
Q27	Explain how MongoDB's indexing and aggregation features enhance query performance.	6
Q28	Discuss the real-world applications where MongoDB is more suitable than RDBMS.	6
Q29	Compare MongoDB's document-based storage with traditional table-based storage.	6
Q30	Justify the need for MongoDB in cloud-based applications.	6
Q31	Describe the purpose of MongoDB's aggregation framework and name two common pipeline stages.	4